



Sass. Part 1.

SCSS



Variables

Много понятий пересекаются в разных препроцессорах. Чаще всего отличаются синтаксисом + возможны некоторые вариации в поведении\функционале.

Переменные в SCSS:

```
$border-width 2px;
```

```
.block{
```

```
border: $ border-width solid red;
```

```
}
```



Variables

Советы для вынесения значений в переменные:

- значение используется по крайней мере дважды
- значение возможно будет изменено в будущем
- Все места, где используется значение, связаны с переменной (т.е. не случайно имеют одинаковое значение)

Затенение переменной - когда в рамках **локальной области видимости** мы объявляем переменную, которая уже есть в **глобальной области видимости**. Тогда переменная будет иметь новое значение только в этой локальной области видимости. Если же мы хотим глобально с локальной области видимости переназначить переменную - существует правило !global

```
$base-color: black;
```

```
$base-color: red !global;
```



Variables: Lists

List - один из типов переменных, для объединения переменных в общие группы. Sass предоставляет набор утилит для работы с list.

Синтаксис: `$pages: (`

`'index',`

`'gallery',`

`'contacts',`

`'shop'`

`);`



Variables: Lists - methods

`length($pages)` -> 4 - узнать длину списка

`nth($pages, 1)` -> 'index' - обратиться к элементу по индексу в списке (индекс начинается с 1 и возрастает с каждым элементом)

`index($pages, 'gallery')` -> 2 - узнать индекс элемента, если он есть в массиве, иначе спец. значение - null

`append($list, $value)` - добавление в массив новых элементов.

`join($list1, $list2)` - объединение двух списков



Variables - maps

Мапы (ассоциативные массивы), пара: ключ и значение, где ключи используются для поиска значений. Они позволяют легко собирать значения в именованные группы и получать доступ к этим группам динамически.

Синтаксис: \$map: (

key: value,

other-key: other-value

);

ВАЖНО! Ключи должны быть уникальными. Ключи могут быть не только строками, но и листами, другими мапами и другими типами, которые есть в Sass



Variables - maps

Чтобы достать значение по ключу - есть функция `map-get`:

```
map-get($map, $key);
```

Чтобы узнать, есть ли у карты значение по ключу - `map-has-key($map, $key)`



Extend

Если мы хотим переиспользовать одни стили во многих местах - можно использовать extend.

Так мы говорим, что хотим в одном селекторе переиспользовать какие-то стили, и , возможно расширить их.

```
.btn {  
  
  outline: none;  
  
  border: none;  
  
  background: grey;  
  
}  
  
.btn-warning {  
  
  @extend .btn;  
  
}
```




Extend

Советую использовать `extend` в тех местах, когда элементы действительно логически взаимосвязаны, и, вдруг, главные стили, от которых расширяли другие стили - поменяется, то логичной будет смена во всех связанных местах. Если же хочется переиспользовать стили в абсолютно несвязанных местах - `extend` не подходит.

Еще важная особенность - `extend` не поддерживает аргументы + не может использоваться в `@media` запросах



Mixins

Миксины - аналог миксинов в LESS. Существуют для переиспользования стилей.

Могут быть без аргументов, с аргументами, и с аргументами и их значениями по умолчанию

```
@my-mixin($color: red) {
```

```
background: $color;
```

```
}
```

```
.container {
```

```
@include my-mixin(black);
```

```
}
```



Mixins

Если у нас есть миксин с множеством аргументов, мы можем передавать туда листы и мапы:

```
$border: (1px solid red);
```

```
@mixin bordered($width, $style, $color) {
```

```
border: $width $style $color);
```

```
}
```

```
.container {
```

```
@include bordered($border...);
```

```
}
```